

Laporan Praktikum 4 Kontrol Cerdas

Nama : Muhamad Azril Insanul Huda

NIM : 224308012

Kelas : TKA-6A

Akun Github (Tautan) : <https://github.com/azl1254>

Student Lab Assistant : Muhammad Mahirul Faiq (214308043)

1. Judul Percobaan

Reinforcement Learning For Autonomous Control

2. Tujuan Percobaan

Tujuan dari percobaan praktikum Reinforcement Learning yaitu:

1. Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
2. Mengimplemtasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN).
3. Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom

3. Landasan Teori

Reinforcement Learning adalah proses mapping (pemetaan) dari situasi yang ada di environment (states) ke bentuk aksi (behavior) agar dapat memaksimalkan reward. Agent yang bertindak sebagai sang learner tidak perlu diberitahukan behavior apakah yang akan sepatutnya dilakukan, atau dengan kata lain, sang learner belajar sendiri dari pengalamannya. Salah satu konsep utama dalam Reinforcement Learning yaitu Agent. Dalam Reinforcement Learning (RL), Agent adalah entitas atau program yang mengambil keputusan dan berinteraksi dengan lingkungan untuk mencapai tujuan tertentu. Agent mengambil tindakan, mendapatkan feedback, dan belajar dari pengalaman untuk meningkatkan performanya. Tugas utama dari Agent yaitu mengamati lingkungan, memilih tindakan atau action, menerima feedback, dan memperbaiki strategi. Maka dari itu Agent adalah inti dari Reinforcement Learning yang bertindak berdasarkan pengamatan lingkungan, menerima reward, dan terus belajar untuk meningkatkan kemampuannya.

Python cv2 dan numpy adalah pustaka pemrograman yang digunakan untuk visi komputer secara waktu nyata. Cv2 berfungsi sebagai pustaka pengolahan citra untuk berbagai jenis objek, baik objek manusia maupun objek benda lainnya, sedangkan numpy digunakan untuk komputasi dan

perhitungan numerik. Dengan menggabungkan kedua pustaka ini, proses deteksi objek pada citra dapat dilakukan dengan lebih cepat dan akurat melalui iterasi numerik yang efisien.

Deep Q-Network (DQN) adalah algoritma dalam reinforcement learning yang menggabungkan Q-learning dengan jaringan saraf dalam untuk menyelesaikan masalah kompleks, sehingga memungkinkan agen untuk belajar dari pengalaman dan membuat keputusan dalam lingkungan yang rumit. Algoritma ini sangat berguna untuk menangani masalah dengan ruang keadaan yang besar, seperti pengenalan gambar dan kontrol robot, dengan memanfaatkan kemampuan jaringan saraf untuk mengekstrak fitur dari data secara efisien dan efektif. DQN bekerja dengan cara memanfaatkan jaringan saraf untuk memperkirakan nilai Q, yang merupakan nilai dari setiap aksi yang mungkin diambil dalam suatu keadaan tertentu. Dengan menggunakan pemutaran ulang pengalaman, DQN menyimpan pengalaman masa lalu dalam memori dan mengambil sampel acak dari pengalaman tersebut untuk memperbarui model, sehingga meningkatkan stabilitas dan efisiensi pembelajaran. Selain itu, DQN juga menerapkan strategi eksplorasi epsilon-greedy, yang memungkinkan agen untuk menyeimbangkan antara eksplorasi tindakan baru dan eksploitasi tindakan yang sudah diketahui, sehingga agen dapat menemukan strategi yang optimal seiring waktu.

4. Analisis dan Diskusi

MountainCar-v0 adalah lingkungan di mana sebuah mobil harus mencapai puncak bukit meskipun daya yang dimilikinya tidak cukup untuk langsung mencapainya. Lingkungan ini memiliki ruang keadaan dua dimensi, yaitu posisi mobil di sepanjang bukit dan kecepatannya. Agen dapat memilih salah satu dari tiga tindakan: bergerak ke kiri, diam di tempat, atau bergerak ke kanan. Untuk mencapai tujuan, agen harus menentukan aksi yang paling efektif guna mengoptimalkan pergerakan mobil.

Dalam lingkungan ini, agen memperoleh reward standar sebesar -1 untuk setiap langkah yang diambil, yang mendorongnya menyelesaikan tugas secepat mungkin. Namun, dalam beberapa versi, reward ini dapat disesuaikan guna mempercepat proses pembelajaran. Tantangan utamanya adalah mobil harus berayun maju-mundur untuk mengumpulkan momentum yang cukup agar bisa mencapai puncak.

Reinforcement Learning (RL) digunakan untuk melatih agen dalam lingkungan ini. Agen belajar dengan berinteraksi dengan lingkungan, mencoba berbagai tindakan, dan menerima umpan balik dalam bentuk reward atau punishment. Dengan menerapkan algoritma RL, agen dapat mengembangkan strategi optimal untuk mencapai tujuan, belajar dari pengalaman sebelumnya, dan menyesuaikan diri dengan perubahan kondisi. Karena sifat lingkungan yang kontinu, pendekatan Deep Q-Network (DQN) menjadi pilihan yang efektif. DQN mampu menangani ruang keadaan yang kompleks dan besar dengan memanfaatkan jaringan saraf untuk memperkirakan nilai Q, sehingga meningkatkan efisiensi agen dalam mencapai puncak.

5. Assignment

Pada praktikum ini, dilakukan implementasi algoritma Deep Q-Network (DQN) untuk melatih agen dalam lingkungan MountainCar-v0. Tujuan utama agen adalah mencapai puncak bukit dengan memilih aksi yang paling optimal. Proses ini diawali dengan mengimpor beberapa pustaka penting, seperti *gymnasium* untuk lingkungan reinforcement learning, *numpy* untuk operasi numerik, *random* untuk pemilihan acak, serta *deque* dari *collections* untuk menyimpan pengalaman. Selain itu, *tensorflow.keras* digunakan dalam pembangunan model jaringan saraf.

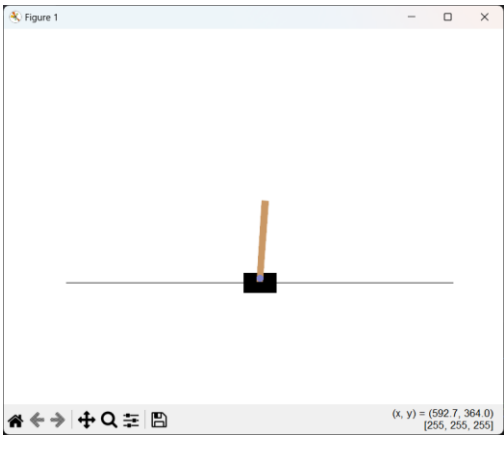
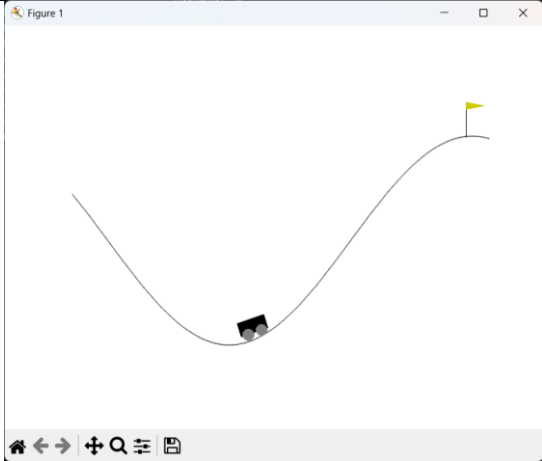
Agan DQN didefinisikan dalam kelas *DQNAgent*, yang diinisialisasi berdasarkan ukuran keadaan dan aksi. Beberapa parameter utama yang digunakan antara lain *gamma* sebagai faktor diskon, *epsilon* untuk eksplorasi, dan *learning_rate* untuk mengoptimalkan model.

Dalam eksperimen pertama pada lingkungan *CartPole-v1*, agen bertugas menjaga keseimbangan tiang di atas gerobak. Karena lingkungan ini relatif sederhana dan dapat diprediksi dengan baik, agen

dapat belajar dengan cepat dan memperoleh reward setiap kali berhasil mempertahankan keseimbangan tiang. Pelatihan pada *CartPole-v1* biasanya dapat diselesaikan hanya dalam beberapa ratus episode.

Sebaliknya, pada *MountainCar-v0*, agen harus menggerakkan mobil hingga mencapai puncak bukit, meskipun mobil tidak memiliki tenaga yang cukup untuk langsung mencapainya. Agen harus mempelajari strategi membangun momentum dengan mengayun maju dan mundur. Tantangan dalam lingkungan ini lebih kompleks, karena reward lebih sulit diperoleh jika mobil gagal mencapai puncak. Oleh karena itu, pelatihan untuk *MountainCar-v0* membutuhkan lebih banyak episode dibandingkan *CartPole-v1*, serta memerlukan arsitektur model yang lebih kompleks atau teknik tambahan untuk meningkatkan stabilitas. Nilai yang diperbarui dalam lingkungan ini cenderung berfluktuasi sebelum akhirnya mencapai skor yang lebih tinggi.

6. Data dan Output Hasil Pengamatan

No	Variabel	Hasil Pengamatan	
		Cartpole	Mountain car
1	Hasil Score (DQN)	<pre>Episode: 952/1000, Score: 234, Epsilon: 0.01 Episode: 953/1000, Score: 229, Epsilon: 0.01 Episode: 954/1000, Score: 182, Epsilon: 0.01 Episode: 955/1000, Score: 171, Epsilon: 0.01 Episode: 956/1000, Score: 153, Epsilon: 0.01 Episode: 957/1000, Score: 139, Epsilon: 0.01 Episode: 958/1000, Score: 151, Epsilon: 0.01 Episode: 959/1000, Score: 175, Epsilon: 0.01</pre>	<pre>Episode: 985/1000, Score: 199, Epsilon: 0.01 Episode: 986/1000, Score: 199, Epsilon: 0.01 Episode: 987/1000, Score: 199, Epsilon: 0.01 Episode: 988/1000, Score: 199, Epsilon: 0.01 Episode: 989/1000, Score: 199, Epsilon: 0.01 Episode: 990/1000, Score: 199, Epsilon: 0.01 Episode: 991/1000, Score: 199, Epsilon: 0.01</pre>
2	Hasil Score (Test)	<pre>Test Episode: 1, Score: 14 Test Episode: 2, Score: 8 Test Episode: 3, Score: 11 Test Episode: 4, Score: 9 Test Episode: 5, Score: 9</pre>	<pre>Test Episode: 1, Score: 199 Test Episode: 2, Score: 199 Test Episode: 3, Score: 199 Test Episode: 4, Score: 199 Test Episode: 5, Score: 199</pre>
3	Hasil Run		

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

1. Penggunaan target network membantu stabilitas pembelajaran, menghindari perubahan nilai Q yang terlalu drastis.
2. MountainCar-v0 lebih sulit dipelajari karena *reward* hanya diberikan saat mencapai puncak bukit, memerlukan strategi eksplorasi lebih lama, dan mengandalkan momentum untuk

berhasil.

3. Cartpole lebih cepat dipelajari karena agen dapat menerima reward setiap Langkah yang menjadikannya lebih optimal

8. Saran

Untuk meningkatkan performa agen DQN, dapat diterapkan *Target Network* untuk meningkatkan stabilitas dan menghindari overfitting terhadap pola jangka pendek. Hal ini dilakukan dengan memperbarui *Target Network* secara berkala. Selain itu, strategi *epsilon decay* yang lebih adaptif, seperti menyesuainya berdasarkan kinerja agen, dapat membantu mencapai keseimbangan yang lebih baik antara eksplorasi dan eksploitasi. Optimalisasi hiperparameter, seperti *gamma*, *learning rate*, dan *epsilon*, juga dapat meningkatkan efektivitas pembelajaran agen.

9. Daftar Pustaka

Angelina, L., Purwanto, Y., & Novianty, A. (n.d.). *Pengambilan Keputusan Pada Trafik Management Dengan Menggunakan Reinforcement Learning Decision Making on Traffic Management Using Reinforcement Learning*

Baradja, A., & Tjendrowasono, T. I. (2024). Pengaplikasian Deep Reinforcement Q-Learning Untuk Prediksi Perdagangan Valas Otomatis. *Jurnal Rekayasa Sistem Informasi dan Teknologi*, 1(3), 190–198. <https://doi.org/10.59407/jrsit.v1i3.519>

Ahmad Husna Ahadi et al. (2024). *Implementasi sistem pendeteksi warna objek dengan opencvpython*